



Model layer - the pieces of the game

Tile **MODEL**

src/tile.h/.cpp

One domino: two ends of 0-6 pips. The building block everything else is made of - a hand, the boneyard and the table are all just tiles. Never prints or reads.

TALKS TO

Used by Hand, Boneyard, Layout, Move, Computer. Calls nothing - it sits at the bottom of the dependency chain.

```
Tile(l, r)
    Builds a tile; out-of-range values leave it as blank 0-0.
getLeftPips() getRightPips()
    The two pip counts.
getSum()
    Left + right. Decides who starts; scores the bonus.
isDouble()
    Both ends match. Doubles become the spinner and count twice at an open end.
flipped()
    A copy with ends swapped (2-5 to 5-2); the original is untouched.
toString()
    Text form "2-5", for screen and save file.
operator==
    Same domino either way round, so 2-5 == 5-2.
setPips(l, r)
    Changes both ends; false and no change if out of range.
isValidPips(p)
    static - is p a legal pip count (0-6)?
fromString(text, out)
    static - parses "2-5"; false on bad text. Guards keyboard and file input.
```

Hand **MODEL**

src/hand.h/.cpp

The tiles one player holds. Owns a vector<Tile> and keeps it sensible: no duplicates, no removing what is not there. Handed out by value so nobody can alter a player's tiles behind their back.

TALKS TO

Owned by Player. Read by Round (deal, sums, empty check), Computer (candidate moves), Human (contains), Serializer/GameState (save). Printed by View.

```
Hand()
    Empty hand.
Hand(const Hand&) operator=
    Copy ctor and assignment - it owns an aggregate.
size() isEmpty()
    Count, and whether the player has gone out.
getTile(i)
    Copy of tile i; blank 0-0 if out of range, not a crash.
getTiles()
    Copy of every tile, for looping.
getSum()
    Total pips - compared to award the lighter-hand bonus.
contains(tile)
    Flip-insensitive search.
toString()
    "0-1 2-5 4-4" in save-file form.
addTile(t)
    Adds; false if already held, so a bad file cannot duplicate a domino.
removeTile(t)
    Removes; false if not held.
clear()
    Empties the hand for a new round.
indexOf(t)
    private - position or -1; shared by contains and removeTile.
```

Boneyard **MODEL**

src/boneyard.h/.cpp

The shuffled stock not yet dealt. Draws come off the front (left-most in the file), returns go on the bottom, so file order is exactly draw order.

TALKS TO

Owned by Round, which resets, shuffles, deals and draws from it. Holds Tiles. Saved as text in GameState.

```
FULL_SET_SIZE
    28 - a double-six set (7+6+5+4+3+2+1).
Boneyard()
    Empty; call reset() for a full set.
copy ctor / operator=
    Copy every tile - it owns an aggregate.
size() isEmpty()
    How many remain.
peekFront()
    The next tile to be drawn, without taking it.
getTiles() contains(t)
    Copy of the stock; is a tile still in it?
toString()
    "0-6 1-5 5-6" in save-file form.
reset()
    Fills with all 28 tiles, 0-0 through 6-6.
shuffle()
    Randomises the order.
drawTile(out)
    Takes the front tile; false when empty - that false is how a pass is detected.
addToBottom(t)
    Puts a tile back; false if already present.
setTiles(list)
    Replaces the contents when resuming; false if the list has duplicates.
clear()
    Removes every tile.
hasDuplicates(list)
    private static - the check behind setTiles.
```

Move **STRUCT**

src/move.h

One placement: which tile on which end. A plain record so a decision travels as a single value. The valid flag lets a function say "no move possible" by returning a Move rather than throwing.

TALKS TO

Returned by every Player::chooseMove. Built by Human and Computer, consumed by Layout::place and Round.

```
tile
    The tile to place.
end
    'L', 'R', 'U' or 'D'.
valid
    False means no move; everything that takes a Move checks this first.
Move()
    The blank invalid move - what Computer::recommend returns when nothing is legal.
Move(tile, end)
    A real move; sets valid = true automatically.
```

The table, and the players who use it

Layout MODEL

src/layout.h/.cpp

The tiles on the table and every rule about where a tile may go - the rule-heaviest class in the program. A main line runs Left to Right; the first double becomes the spinner and opens the Up and Down arms. Tiles are stored already oriented so touching pips match.

TALKS TO

Owned by Round. Queried by Player::canPlay, Human (open ends, error text) and Computer (copies it to test a move). Drawn by View::showLayout.

LEFT RIGHT UP DOWN

The four end letters, used instead of bare characters.

NO_SPINNER

-1: no double played yet; also returned for a closed end.

copy ctor / operator=

Copy every tile. Computer depends on these to try a move on a throwaway copy.

isEmpty() hasSpinner()

Nothing played yet; a double has been played so U and D exist.

getLine() getUpArm() getDownArm()

Copies of the three runs, arms running outward from the spinner.

getSpinnerIndex()

Where the spinner sits on the line, or NO_SPINNER.

isOpenEnd(end)

May tiles go here now? U and D are closed until a spinner exists.

getEndPips(end)

Pips showing there, or -1 if closed.

getOpenEndSum()

Sum across all open ends, a double counting BOTH its pips. Round scores this; Computer tries to leave it small.

canPlace(tile, end) / (move)

The single source of truth for legality.

place(tile, end) / (move)

Places it, flipping if needed, and promotes a first double to spinner. False and no change if illegal.

toString() fromString(text)

"L 5-3 [D 3-3 U] 3-2 2-2 R" out and back in; fromString returns false on invalid text.

clear()

Empties the table.

endName(end) isEndLetter(end)

static - "Left"/"Right"/"Up"/"Down"; is this one of the four letters?

spinnerPips() endValue(t, exposed)

private - what the arms start from; what a tile shows at an end (both pips for a double).

orientChain(tiles)

private static - flips tiles so each right pip meets the next left pip; false if nothing connects. This validates a layout read from a file.

Human PLAYER SUBCLASS

src/human.h/.cpp

The human player: turns keyboard input into a legal Move and refuses to return until it really is legal. Adds no data - just a Player named "Human" that asks instead of computes.

TALKS TO

Calls Computer::recommend for help mode, View for all I/O, Tile::fromString to parse, Layout to validate. Called by Round, which does not know it is a Human.

Human()

A Player named "Human".

chooseMove(layout, target)

Loops until the user names a tile they hold that the layout accepts at an end they pick. Rejects non-tiles and tiles not in hand with a different message for each. Typing help (or h) prints the computer's suggestion and reason, then asks again - help costs nothing and commits to nothing.

askEnd(layout, tile)

private - returns 'L' without asking on an empty table; otherwise lists only the currently open ends, accepts either case, and re-asks until the answer is one of them.

Player ABSTRACT BASE

src/player.h/.cpp

The base class for both players. Name, hand, round score and tournament score live here; only the decision is left to the subclass. chooseMove is pure virtual, so Round can hold Player* and never ask which kind it has.

TALKS TO

Owns a Hand, queries Layout, returns a Move. Subclassed by Human and Computer. Driven by Round; owned by Tournament.

Player(name)

Name (default "Player"), both scores zero.

~Player()

Virtual - required, since players are held through a base pointer.

getName()

Used in every message and as the save-file key.

getHand()

A copy - the caller cannot change the player's tiles through it.

getHandSum() hasTiles()

Pips held; false once the hand is empty.

getRoundScore() getTournamentScore()

This round, and everything before it.

canPlay(layout)

Can any held tile go anywhere right now? Round uses this to force a draw or a pass.

addTile removeTile setHand clearHand

Forwarded to the Hand; setHand is for resuming.

addRoundPoints(n)

Adds points; false for a negative amount.

setScores(round, tourn)

Sets both directly when resuming; false if either is negative.

finishRound()

Folds the round score into the tournament score and zeroes it.

chooseMove(layout, target) = 0

Pure virtual. Round guarantees at least one tile is playable before calling, so no subclass handles "nothing fits".

Computer PLAYER SUBCLASS

src/computer.h/.cpp

The computer player and the home of the strategy. The reasoning sits in the static recommend() so Human's help mode calls the identical code - there is one strategy in the program, not two. chooseMove is a thin wrapper that announces the result.

TALKS TO

Reads a Hand and a Layout, returns a Move, speaks through View. recommend() is also called by Human.

Computer()

A Player named "Computer".

chooseMove(layout, target)

Calls recommend, prints the tile, the place and the reason, returns the Move.

recommend(hand, layout, target, reason)

static - the whole strategy. Lists every legal (tile, end) pair, keeping one per tile on an empty table. 1st: the highest-scoring move, ties to a double then the heavier tile. 2nd (nothing scores): the heaviest double, at the end leaving the smallest open-end total - doubles are hard to shed and count twice at the end. 3rd (no double): the heaviest tile, same end rule, to lighten the hand and starve the opponent. Fills reason with a plain-English sentence; returns an invalid Move if nothing is legal.

pointsFor(layout, move, target)

private - the resulting open-end sum if it is a multiple of the target, else 0.

sumAfter(layout, move)

private - copies the Layout, plays the move on the copy, returns the open-end sum. The real table is never touched while the computer thinks.

Running the game, and talking to the outside

Round CONTROLLER

src/round.h/.cpp

One round from deal to final score - the referee. It owns the table and the stock, tracks whose turn it is, enforces drawing and passing, awards points and ends the round. Holds its players as Player*, so a Human and a Computer are driven through identical calls.

TALKS TO

Owns Layout and Boneyard; drives two Player*; reports through View; converts to and from GameState. Created and run by Tournament.

PLAYER_COUNT HAND_SIZE
2 players, 7 tiles each.

MIN_TARGET MAX_TARGET
3 and 5 - the only legal round targets.

enum Outcome
FINISHED or SAVE_REQUESTED - how play() tells Tournament why it returned.

Round(first, second, target)
Stores the two pointers; first draws first when deciding who starts.

getTargetScore() getLayout() getBoneyard()
The target, and copies of the table and stock.

getNextPlayerName()
Who is about to move - written into the save file.

toState(tournScore, state)
Copies the round into a GameState: target, layout text, boneyard text, next player, and each player's hand and both scores keyed by name.

startNew()
Reset and shuffle the stock, clear table and hands, deal, pick the first player.

restore(state)
Rebuilds layout and boneyard from text, checks both players appear and the named next player is one of them, then hands out saved hands and scores. False and no change if any of that fails.

play()
The turn loop: show the table, take a turn, swap players, and after each successful placement offer to save and quit.

dealHands()
private - seven tiles from the stock to each player.

chooseFirstPlayer()
private - both draw one tile, higher total starts, both tiles go back on the bottom, repeat on a tie.

takeTurn(player)
private - draw until able to play (pass if the stock empties), ask for the move, place it, remove it from the hand, and award the open-end sum as points when it is a multiple of the target. False means passed.

finish()
private - show both hands, give the lighter hand (heavier - lighter) / target as a bonus, print totals, fold round scores into tournament scores.

showTable()
private - layout, both hands, the stock and both scores before each turn.

Tournament CONTROLLER (TOP)

src/tournament.h/.cpp

The whole session. The only class that knows a Human and a Computer exist as concrete types, and the only one that touches the file system. Plays rounds until somebody reaches the tournament score.

TALKS TO

Owns a Human and a Computer, passes them to Round as Player*. Calls Serializer to load and save, View for every prompt. Called only by main().

MIN/MAX_TOURNAMENT_SCORE
1 and 10000 - the range View will accept.

Tournament()
Creates both players; tournament score zero.

run()
The entire program flow: offer to resume, get the tournament score, then loop - build a Round, restore it or ask for this round's target (3 or 5) and deal fresh, play it. SAVE_REQUESTED writes the file and stops; otherwise print running totals and check for a winner. A save file that does not describe a valid game prints a message and starts fresh rather than crashing.

askResume(state)
private - asks whether to resume, then for a file name, offering another name if the load fails and returning false if the player gives up.

saveAndQuit(round)
private - copies the round into a GameState and asks for a file name until the write succeeds.

isOver() announceWinner()
private - has anyone reached the target; print both totals and name the winner, or a draw.

Serializer UTILITY (ALL STATIC)

src/serializer.h/.cpp

Reads and writes the course's text save format - a pure translator between a GameState and a file. Holds no data. Because it is the only class that opens a file, the format is described in exactly one place.

TALKS TO

Fills and empties a GameState, builds Hands from text, parses Tiles. Called only by Tournament.

save(state, fileName)
Writes the sections in the required order: tournament score, target score, each player's block (hand, current score, tournament score), layout, boneyard, next player. The Computer's block comes before the Human's, matching the course example. False if the file will not open or the stream goes bad.

load(fileName, state)
Reads line by line, remembering which player section it is in - a bare "Name:" opens one. Resolves "Tournament Score:" by context: outside a section it is the tournament target, inside it is that player's total. False on a bad number, a malformed tile, a duplicate in a hand, an unrecognised line, or a next player with no section.

parseHand(text, hand)
private - fills a Hand from "0-1 2-5 4-4"; false on a bad or repeated tile.

trim(text)
private - copy without leading or trailing whitespace.

startsWith(line, prefix, rest)
private - if the line has the prefix, puts the trimmed remainder in rest. This is what turns each labelled line into a field.

GameState STRUCTS

src/gamestate.h

Everything needed to save a game and pick it up later: PlayerState for one player, GameState for the whole save. They exist so Serializer and Round never have to know about each other - Round fills and consumes one, Serializer writes and reads one.

TALKS TO

Holds Hands. Passed between Round, Serializer and Tournament. Calls nothing itself.

PlayerState: hand, roundScore, tournamentScore
One player's tiles and both scores.

tournamentScore
The score the tournament is played to.

targetScore
3 or 5 for the round being saved.

players
map<string, PlayerState> keyed by name - which is how restore() checks its players appear.

layoutText boneyardText
The table and the stock as text, because the save file is text. Layout and Boneyard each know their own format, so the record stays a dumb carrier.

nextPlayer
Name of the player to move.

PlayerState() GameState()
Default ctors zero everything, so a fresh record is always in a known state.

View NAMESPACE

src/view.h/.cpp

All screen output and keyboard input. A namespace, not a class, because there is nothing to store. Its reason for existing: because View is here, Tile, Hand, Boneyard and Layout contain no cin or cout at all.

TALKS TO

Reads Layout and Hand to display them. Called by Human, Computer, Round and Tournament. Nothing calls back into the game from here.

message(text) blankLine()
Every line the program prints goes through here, so the output style lives in one function.

showLayout(layout)
ASCII picture of the table: the main line with L and R labelled, the arms off the spinner, doubles drawn cross-wise.

showHand(owner, hand)
"Human's hand: 0-1 2-5 4-4 (sum 16)".

readInt(prompt, min, max)
Will not return a number outside the range - tournament score, round target.

readYesNo(prompt)
Will not return until it gets a yes or a no - resume, save and quit.

readWord(prompt)
One word: a tile, help, an end letter, a file name.

readLine(prompt)
A whole line, spaces included.